OOP Report: Vehicle Rental Management System

Name: Dania Alkhmous | **Course:** Object-Oriented Programming | **Assignment:** Vehicle Rental Management System

1. Introduction

In this assignment I built a console application in Java for a vehicle rental company. The company rents cars, motorcycles and trucks. The program manages vehicles, customers, rentals, returns, maintenance and revenue. The main goal was not only to make a working menu, but to use the main ideas of object-oriented programming (OOP) in a meaningful way. In this report I explain abstraction, abstract classes versus interfaces, the two forms of polymorphism, overloading and overriding, and I give examples from my project.

2. Abstraction

Abstraction means showing only what is important and hiding the details. In my project, the `Vehicle` class is an abstraction. The rest of the program knows that every vehicle can calculate a rental cost, but it does not need to know how each type does it. For example, `RentalSystem` just writes:

```
double total = vehicle.calculateRentalCost(days, optionalService);
```

It does not care if the vehicle is a car, a motorcycle or a truck. Each type hides its own calculation inside its own class. Abstraction also makes the code easier to change: if I add a new type of vehicle, `RentalSystem` does not need to change.

3. Abstract Class versus Interface

An **abstract class** is a parent class that cannot be created with `new`. It can have fields, constructors, normal methods and abstract methods (methods without a body). In my project, `Vehicle` is abstract. It has private fields like `brand` and `dailyPrice`, constructors, a normal method `calculateBaseCost`, and three abstract methods: `calculateRentalCost`, `displayInfo` and `getOptionalServiceName`. Every subclass must write its own version of these abstract methods.

An **interface** is a contract. It only lists the methods that a class must have, without code. My interface `Maintainable` has two methods: `performMaintenance()` and `needsMaintenance()`. `Car`, `Motorcycle` and `Truck` all implement it, and each one writes its own maintenance behavior.

	Abstract class (`Vehicle`)	**Interface (`Maintainable`)**
Meaning	What the object **is** (is-a)	What the object **can do** (can-do)
Fields	Yes	No (only constants)
Constructors	Yes	No
Methods with code	Yes	No (only method signatures)
Keyword	`extends`	`implements`
How many can a class use?	Only one	Many

I needed both in this project. `Vehicle` holds the data and code that all vehicles share (a vehicle **is** a vehicle). `Maintainable` describes an ability (a vehicle **can be** maintained). Because the maintenance ability is separate from `Vehicle`, it can be used by any other class in the future, even if it is not a vehicle.

4. Polymorphism

Polymorphism means "many forms". The same method call can do different things depending on the real object. I used it in two ways.

4.1 Polymorphism with inheritance (Vehicle references)

I store all vehicles in one list: `ArrayList<Vehicle>`. The list can contain cars, motorcycles and trucks together. When I call `vehicle.calculateRentalCost(days, optionalService)`, Java runs the version that belongs to the real object at runtime:

- Car: $60/day, 4 days, with insurance = (60 x 4) + (15 x 4) = **$300**
- Motorcycle: $30/day, 2 days, with helmet = (30 x 2) + (5 x 2) = **$70**
- Truck: $100/day, 3 days = (100 x 3) + 50 = **$350**

I did not use any `if/else` or `switch` on the vehicle type. The same is true for `displayInfo()`, which prints different details for each type.

4.2 Polymorphism with interfaces (Maintainable references)

The second form uses the interface. In `RentalSystem`, I look at the vehicles through a `Maintainable` reference:

```
if (v instanceof Maintainable) {
    Maintainable m = (Maintainable) v;
    if (m.needsMaintenance()) { ... }
}
```

and later `m.performMaintenance()`. Here I do not care about the exact type. I only care that the object can be maintained. The result is different for each type: a car prints that the oil, brakes and tires were checked; a motorcycle prints that the chain and engine were checked; a truck prints that the engine, brakes and cargo area were inspected.

The difference between the two forms is the "view" of the object. In the first form I see it as a `Vehicle` (what it is). In the second form I see it as a `Maintainable` (what it can do).

5. Overloading and Overriding

Overloading means writing several methods (or constructors) with the **same name but different parameters** in the same class. Java chooses the correct one at compile time. In my project:

- `searchVehicle(String vehicleId)` and `searchVehicle(String brand, String model)` in `RentalSystem`: one searches by ID, the other by brand and model.
- Constructors: `Car` can be created with 7 parameters (including seats and transmission) or with 5 parameters (using default values: 5 seats and automatic). The short constructor calls the long one with `this(...)`.

Overriding means a subclass writes its **own version of a method** that already exists in the parent class, with the **same name and the same parameters**. Java chooses the version at runtime, based on the real object. In my project, `Car`, `Motorcycle` and `Truck` override `calculateRentalCost` and `displayInfo` from `Vehicle`, and they write the methods of the `Maintainable` interface. For example, `Truck` overrides the cost calculation to add the fixed $50 service fee, and it ignores the optional service parameter because trucks have no optional service. I used `@Override` so the compiler checks that I really override a method.

	Overloading	**Overriding**
Where	Same class	Parent and child class
Parameters	Different	Same
Decided	At compile time	At runtime
Example	`searchVehicle(id)` / `searchVehicle(brand, model)`	`calculateRentalCost` in `Car` and `Truck`

6. Conclusion

In this project I used abstraction, inheritance, abstract classes, interfaces, two forms of polymorphism, overloading and overriding together. The abstract class `Vehicle` gave me shared code and a common type. The interface `Maintainable` gave me a separate ability that works through its own reference. Polymorphism helped me avoid long `if/else` chains, and the code is easier to extend: adding a new vehicle type needs only a new subclass. I also learned that good design is more important than only having a working menu.